

Arrays and Vectorized Computation

Engineering Computation with Python – Chapter 6

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

- 1 **One Value, Then Many**
- 2 **Creating and Accessing Arrays**
- 3 **Slicing and Subscripting**
- 4 **Element-wise Operations**
- 5 **Vectorizing Formulas**
- 6 **Accumulation and Boolean Masks**
- 7 **Summary**

1 One Value, Then Many

2 **Creating and Accessing Arrays**

3 **Slicing and Subscripting**

4 **Element-wise Operations**

5 **Vectorizing Formulas**

6 **Accumulation and Boolean Masks**

7 **Summary**

A variable like `voltage = 4.7` holds one measurement. Now suppose an instrument logs a reading every 20 ms for two seconds — that is 101 measurements.

Storing each in its own variable does not scale

`voltage0, voltage1, ...` — unworkable past a handful of readings.

A list can hold them, but...

It cannot add 5 to every entry, or convert all 101 values from volts to millivolts, in one step. Each element must be visited **one at a time**, in a loop.

This chapter's central idea

A NumPy **array** stores many values together *and* lets arithmetic apply to **all of them at once**, without writing a loop.

You have written vectors as ordered lists of numbers, and added or scaled them component by component. A NumPy array *is* that vector, stored in memory – the arithmetic you did by hand is what Python performs in a single expression across thousands of elements.

Not needed yet: matrix multiplication, dot and cross products – those arrive in Chapter 9. Here, every operation is element by element.

A NumPy array is the closest thing Python has to a MATLAB matrix. Four differences cause most early mistakes:

- **Indexing starts at zero.** `a[0]`, not `a(1)`; `a[-1]` for the last element.
- **A range excludes its endpoint.** MATLAB's `1:5` gives 5 values; `np.arange(1,5)` gives 4.
- **Multiplication operators are reversed.** In NumPy, `*` is element-wise and `@` is the matrix product — the **opposite** of MATLAB.
- **Shapes are a tuple.** `np.zeros(3)` is a 3-element vector; write `np.zeros((3,3))` for a square one.

- 1 One Value, Then Many
- 2 **Creating and Accessing Arrays**
- 3 Slicing and Subscripting
- 4 Element-wise Operations
- 5 Vectorizing Formulas
- 6 Accumulation and Boolean Masks
- 7 Summary

Lists	array module	NumPy arrays
Ordered, mutable, mixed types. General purpose. <code>[1, 2, 3, 4]</code>	Type-consistent, 1D only. Rarely used directly in this course.	Optimized numerics, multi-dimensional. <code>np.array([1, 2, 3, 4])</code>

Scalar (0D), **1D** (vector), **2D** (matrix), **multi-dimensional** (tensor) — classified by the number of dimensions (axes).

Human Row/Column vs. Python Row/Column

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

Human (1-based)

Row 1, column 1 → value 1.
Row 2, column 3 → value 6.

Python (0-based)

Row 0, column 0 → value 1.
Row 1, column 2 → value 6.

Be precise with language

Always specify which convention you mean when discussing rows and columns.

```
import numpy as np  
a = np.array([[1, 2], [3, 4]])
```

The outer bracket `[ [ ] ]` groups the rows together; each inner bracket `[ ]` is one row.

```
b = np.arange(1, 6, 2)  # like range(), but returns an array
```

Subscript syntax

`array[start:stop:step]` — same rules as list slicing from Chapter 3.

$$a = \begin{pmatrix} 2 & 4 & 1 \\ 10 & 9 & 5 \\ 7 & 8 & 15 \end{pmatrix}$$

Expression	Meaning / Result
<code>a[0,0]</code>	Element at (0,0) $\rightarrow$ 2
<code>a[0]</code>	First row $\rightarrow$ [2, 4, 1]
<code>a[-1]</code>	Last row $\rightarrow$ [7, 8, 15]
<code>a[:,0]</code>	First column $\rightarrow$ [2, 10, 7]
<code>a[[1,2],1]</code>	Rows 1,2, column 1 $\rightarrow$ [9, 8]

Single brackets give a **1D array**; double brackets give a genuine **2D** row or column vector:

```
vector_1d = np.array([1, 2, 3])           # shape (3,)
row_vector = np.array([[1, 2, 3]])        # shape (1, 3)
column_vector = np.array([[1], [2], [3]]) # shape (3, 1)
```

```
[1 2 3]      <- 1D
[[1 2 3]]    <- 2D row
[[1]]
 [2]
 [3]]        <- 2D column
```

1	<u>One Value, Then Many</u>
2	<u>Creating and Accessing Arrays</u>
3	<u>Slicing and Subscripting</u>
4	<u>Element-wise Operations</u>
5	<u>Vectorizing Formulas</u>
6	<u>Accumulation and Boolean Masks</u>
7	<u>Summary</u>

Slicing a List Copies. Slicing an Array Does Not.

One important difference from lists: slicing an array produces a **view** — a window onto the same memory.

```
A = np.array([1, 2, 3, 4])
v = A[1:3]      # a VIEW, not a copy
v[0] = 99
print(A)        # [1 99 3 4]  -- the original changed!

B = np.array([1, 2, 3, 4])
c = B[1:3].copy() # ask explicitly for a copy
c[0] = 99
print(B)        # [1 2 3 4]  -- safe
```

Views make NumPy fast — use `.copy()` when you must modify a slice without disturbing the original.

```
array = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])  
  
array[0, :]      # first row:      [1, 2, 3]  
array[:, 0]      # first column:  [1, 4, 7]  
array[0:2, 1:3]  # 2x2 block:     [[2, 3], [5, 6]]  
array[::2]       # every 2nd row: [[1,2,3], [7,8,9]]
```

What's new vs. list slicing

An array can be sliced in **more than one dimension at once** — a row index and a column index, separated by a comma.

Notation	Meaning
$A[:, c]$	Column c of A
$A[r, :]$	Row r of A
$A[:, :]$	All elements (the entire array)
$A[:, j:k]$	All rows, columns j to $k - 1$
$A[j:k]$	Rows j to $k - 1$
$A[:]$	All elements as a single vector

The colon alone means “all elements.”

linspace and arange

`linspace(start, stop, num)`

Evenly spaced values; num samples; **includes** the endpoint by default.

```
np.linspace(0, 1, num=5)      # [0.  0.25  0.5  0.75  1. ]
np.arange(0, 1.25, 0.25)     # [0.  0.25  0.5  0.75  1. ]
```

`arange(start, stop, step)`

Fixed step size; stop **excluded**, matching `range()`.

`linspace` is often the safer choice for a time axis, since it guarantees the endpoint is included.

- 1 One Value, Then Many
- 2 Creating and Accessing Arrays
- 3 Slicing and Subscripting
- 4 **Element-wise Operations**
- 5 Vectorizing Formulas
- 6 Accumulation and Boolean Masks
- 7 Summary

Arithmetic Is Element-wise

```
A = np.array([[1, 2], [3, 4]])  
B = np.array([[5, 6], [7, 8]])  
  
A + B    # [[6, 8], [10, 12]]  
A - B    # [[-4, -4], [-4, -4]]  
A * B    # [[5, 12], [21, 32]] -- element-wise, NOT matrix product  
A / B    # [[0.2, 0.33], [0.43, 0.5]]  
A ** 2   # [[1, 4], [9, 16]]
```

The MATLAB trap, again

`A * B` is element-wise in NumPy. The matrix product needs `A @ B` (Chapter 9).

Function	Computes	Example
<code>np.sqrt(x)</code>	square root	$\rightarrow 4.0$
<code>np.exp(x)</code>	e^x	$\rightarrow 2.7183$
<code>np.log(x)</code>	$\ln x$	$\rightarrow 1.0$
<code>np.sin(x)</code>	sine (radians)	$\rightarrow 1.0$
<code>np.radians(d)</code>	deg $\rightarrow$ rad	
<code>np.degrees(r)</code>	rad $\rightarrow$ deg	

One call evaluates the function at **every** sample – no loop required.

This causes more wrong answers than any other feature of these functions. Engineering problems are usually *stated* in degrees:

```
print(np.sin(30))           # -0.988 -- 30 RADIANS  
print(np.sin(np.radians(30))) # 0.5    -- 30 degrees
```

No error, just a wrong answer

30 radians is a valid angle — nothing raises an exception. A units check catches this; a computer never will.

Exact zeros are rarely exact

```
print(np.cos(np.pi / 2))  
# 6.123233995736766e-17, not 0.0
```

Never test such a result with ==.

Use arctan2, not arctan, for angles

```
np.degrees(np.arctan(1 / -1))    # -45.0  -- wrong quadrant  
np.degrees(np.arctan2(1, -1))   # 135.0  -- correct
```

`arctan2(y, x)` keeps both signs and returns the correct angle over the full circle.

- 1 One Value, Then Many
- 2 Creating and Accessing Arrays
- 3 Slicing and Subscripting
- 4 Element-wise Operations
- 5 **Vectorizing Formulas**
- 6 Accumulation and Boolean Masks
- 7 Summary

$$B = a(1 + r)^n$$

The same formula, with no changes, works on a single value or an entire array:

```
r, n = 0.09, 10  
  
A1 = 100  
B1 = A1 * (1 + r) ** n      # 236.7364  
  
A2 = np.array([100, 200, 500, 1000, 4000])  
B2 = A2 * (1 + r) ** n      # [236.7, 473.5, 1183.7, ...]
```

This is **vectorization**: apply a formula across an entire array at once, with no explicit loop.

$$s = \frac{gt^2}{2}$$

```
g = 9.81
t = np.arange(0, 6)    # [0, 1, 2, 3, 4, 5]

s = g * t**2 / 2
print(s)
# [0.  4.905 19.62 44.145 78.48 122.625]
```

Every displacement, for every time step, computed in one expression.

EE Focus: The Bench Recording as an Array

101 samples, every 20 ms for two seconds — far too many for individual variables, exactly right for an array:

```
V0, R, C = 12.0, 4700.0, 100e-6
tau = R * C           # 0.47 s

t = np.arange(0, 2.001, 0.02)  # all 101 sample times

v = V0 * np.exp(-t / tau)      # every voltage, one expression
i_mA = (v / R) * 1000          # every current, one expression
```

```
Number of samples: 101
v at t=0: 12.0000 V
i at t=0: 2.5532 mA
v at t=2s: 0.1703 V
```

Three lines of arithmetic produced 202 numbers — worth confirming they are the right ones:

- **Sample count.** 2 s at 20 ms intervals gives 100 intervals, 101 readings. Matches `t.size`.
- **Initial values.** At $t = 0$: $i = 12/4700 \approx 2.55$ mA by Ohm's law. Matches.
- **Decay.** After 2 s (over 4 time constants), voltage has fallen to 0.17 V — close to zero, as expected.

What did not appear above: a loop

Every value was computed for all 101 samples in a single expression — the central reason engineers reach for arrays.

- 1 One Value, Then Many
- 2 Creating and Accessing Arrays
- 3 Slicing and Subscripting
- 4 Element-wise Operations
- 5 Vectorizing Formulas
- 6 Accumulation and Boolean Masks**
- 7 Summary

Arrays make **accumulation** — how much has built up so far — visible early:

```
current = np.array([2.0, 2.0, 1.6, 1.2, 0.8]) # amps, hourly
charge = np.cumsum(current)                 # amp-hours

print(charge) # [2.  4.  5.6 6.8 7.6]
```

This running total is an integral

Chapter 13 develops this properly, alongside its counterpart the **derivative** (a rate of change) — both connect to polynomial models of a system's response.

Relational and logical operators extend naturally to arrays, applying **element-wise**:

```
readings = np.array([24.5, 26.1, 29.8, 22.0, 31.2])  
  
mask = readings > 28  
print(mask)           # [False False  True False  True]  
  
print(readings[mask]) # [29.8 31.2] -- only the "too hot" readings
```

A Boolean mask is a selector

`readings[mask]` returns only the elements where the mask is True — filtering a whole dataset in one line.

- 1 One Value, Then Many
- 2 Creating and Accessing Arrays
- 3 Slicing and Subscripting
- 4 Element-wise Operations
- 5 Vectorizing Formulas
- 6 Accumulation and Boolean Masks
- 7 Summary

- A **NumPy array** stores many values together and applies arithmetic to all of them at once — no loop required.
- Indexing starts at **0**. Slicing uses `[start:stop:step]`, with stop excluded.
- Slicing an array returns a **view**, not a copy — use `.copy()` when you need an independent array.
- NumPy's math functions (`np.sqrt`, `np.sin`, ...) apply **element-wise**; trig functions expect **radians**.

- **Vectorization**: one formula, written once, applies to a single value or an entire array with no change.
- `np.cumsum` previews **accumulation** — Chapter 13's integral.
- A **Boolean mask** (`readings > 28`) selects elements meeting a condition — filtering a dataset in one line.

Next: Data Visualization